

Cho trước những giá trị đang được lưu trữ trong các thanh ghi như bên dưới:

Thanh ghi	Giá trị
%ebp	0xFC
%eax	0x2

Và một đoạn code assembly như bên dưới.

```

1. movl $2, -16(%ebp)
2. movl $3, -12(%ebp)
3. movl $0x10, -4(%ebp)
4. movl -12(%ebp, %eax, 4), %eax
5. sarl $3, %eax
6. movl -16(%ebp), %eax
7. subl $1, %eax
8. movl %eax, (%ebp)

```

Hãy điền vào bảng thông tin **các thanh ghi/ô nhớ bị thay đổi giá trị** và **giá trị mới của nó** sau khi thực thi từng câu lệnh trên?

Lưu ý: Các lệnh thực thi tuần tự, kết quả lệnh thực thi trước có thể ảnh hưởng đến lệnh phía sau. **KHÔNG ĐƯỢC BỎ TRỐNG Ô NÀO** để tránh mất điểm, ô không có thông tin thì điền -

Câu lệnh	Vị trí thay đổi giá trị	Tên thanh ghi/Địa chỉ ô nhớ thay đổi giá trị	Giá trị mới tại vị trí bị thay đổi giá trị	Giải thích
1	Ô nhớ	0xEC	2	Ô nhớ có địa chỉ ($\%ebp - 16$) = $0xFC - 16 = 0xEC$ được gán giá trị hằng số 2
2	Ô nhớ	0xF	3	Ô nhớ có địa chỉ ($\%ebp - 12$) = $0xFC - 12 = 0xF$ được gán giá trị hằng số 3
3	Ô nhớ	0xF8	0x10	Ô nhớ có địa chỉ ($\%ebp - 4$) = $0xFC - 4 = 0xF8$ được gán giá trị hằng số 0x10

4	Thanh ghi	%eax	0x10	Truy xuất ô nhớ có địa chỉ ($\%ebp + \%eax * 4 - 12$) = $0xFC + 0x2 * 4 - 12 = 0xF8$, lấy được giá trị 0x10 gán vào thanh ghi %eax
5	Thanh ghi	%eax	0x2	Lấy giá trị của thanh ghi %eax là 0x10 dịch phải 3 bit tương đương với $0x10 / 2^3 = 0x2$ và gán vào thanh ghi %eax
6	Thanh ghi	%eax	2	Truy xuất ô nhớ có địa chỉ ($\%ebp - 16$) = $0xFC - 16 = 0xEC$, lấy được giá trị 2 và gán vào thanh ghi %eax
7	Thanh ghi	%eax	1	Lấy giá trị của thanh ghi %eax là 2 trừ 1 được 1 và gán vào thanh ghi %eax
8	Ô nhớ	0xFC	1	Truy xuất ô nhớ có địa chỉ %ebp = 0xFC lấy được giá trị 1 và gán vào thanh ghi %eax

Bài tập bonus – Machine programming Basic

Giả sử ta có đoạn mã assembly như bên dưới

```

x tại ô nhớ (%ebp+8), n tại ô nhớ (%ebp+12)
1.  movl 12(%ebp), %ecx    // n
2.  movl 8(%ebp), %edx     // x
3.  xorl %eax, %eax
4.  addl $1, %eax
5.  sall %ecx, %eax
6.  subl $1, %eax
7.  andl %edx, %eax

```

Trả lời các câu hỏi sau:

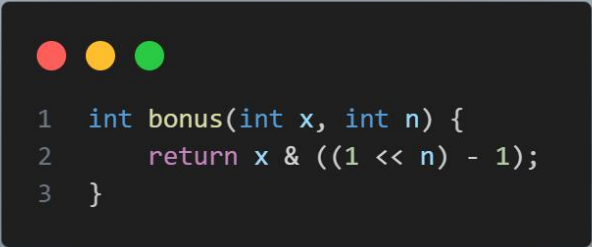
1. Instruction thứ 3 (lệnh **xor**) có tác dụng gì?
2. Instruction thứ 5 thực hiện các phép dịch bit (**sall**) với số bit cần dịch lưu trong thanh ghi %ecx, tuy nhiên đang bị lỗi. Lý giải nguyên nhân bị lỗi và sửa lại cho đúng?

3. Viết hàm C tương ứng với mã assembly trên: **int bonus(int x, int n)**

Thử dự đoán chức năng của đoạn mã này?

Answer:

1. Lệnh xor có tác dụng gán vào thanh ghi %eax có giá trị 0 (%eax = %eax ^ %eax = 0)
2. Lỗi vì các phép toán dịch bit chỉ cho phép toán hạng là hằng số hoặc thanh ghi 8 – bit
Vậy nên sửa lại thành: sall %cl, %eax (%cl có kích thước 8 – bit)
3. Đáp án:



```
1 int bonus(int x, int n) {  
2     return x & ((1 << n) - 1);  
3 }
```